Sistemas Distribuidos · Grado en Tecnologías para la Sociedad de la Información · ETSISI – UPM · 2026-27

Práctica 4 · Marshalling: un protocolo binario con códigos de operación

El servicio «Banco Acme»: de las líneas de texto a los mensajes con tipos

Profesor: David Díaz Pardo de Vera · david.diazp@upm.es · Despacho 4307

Dedicación estimada: 2 h de laboratorio + unas 2 h de trabajo personal · Individual

Material: proyecto Maven esqueleto sd-lab44-skeleton.zip (Moodle) · IntelliJ IDEA (o VS Code / Eclipse) · JDK 17+

---

1. Por qué existe esta práctica

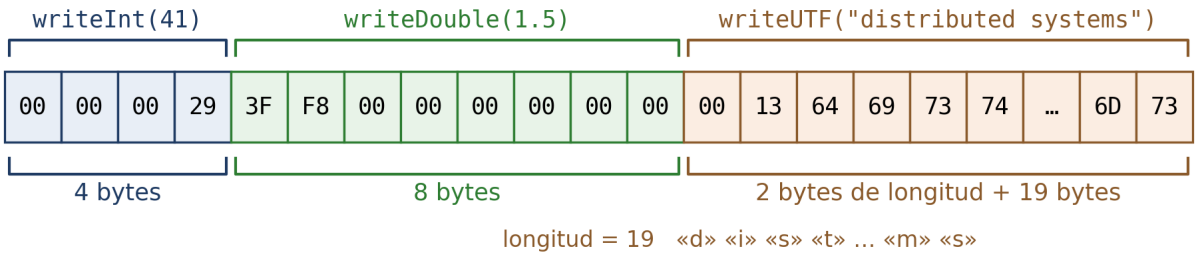
Todo lo que tus servidores han dicho hasta ahora ha viajado como *líneas de texto*: los servidores de eco (prácticas 2 y 3) y el chat (lectura opcional) hablan con `BufferedReader.readLine()` y `PrintWriter.println()`. El middleware real mueve **datos con tipos (enteros, dobles, booleanos, cadenas), y alguien** tiene que convertir esos valores en bytes en un extremo y reconstruirlos en el otro. Esa tarea se llama **marshalling** (y **unmarshalling**), y en esta práctica la harás a mano con `DataOutputStream` / `DataInputStream`, diseñando un protocolo pequeño pero completo organizado en torno a **códigos de operación (opcodes)**.

Éste es el peldaño que faltaba entre los servidores de eco y el Laboratorio 2: allí escribirás proxies de cliente, esqueletos y un ORB en miniatura, y todos descansan exactamente sobre la maquinaria que construyes hoy. Además es materia de examen: **el marshalling forma parte del alcance del examen de prácticas PR-EX-1** (ver la presentación de la asignatura), y en exámenes de laboratorio anteriores se ha pedido precisamente una tabla de opcodes, un método del proxy y el código de despacho del servidor.

> **Una nota sobre el chat.** El enunciado del chat te dice que *no* uses DataInput/DataOutput. Esa instrucción se refiere a elegir bien la representación, y no prohíbe esas clases en general: un chat es una conversación legible por personas, orientada a líneas, y las líneas de texto son el protocolo adecuado para él. Una operación bancaria («ingresa 250,00 en la cuenta 1») son datos estructurados con tipos, y aquí lo adecuado es un protocolo binario. Saber cuál elegir, y por qué, es parte de lo que enseña este peldaño.
>
> **La única idea imprescindible.** Una conexión TCP transporta bytes, no tipos. Nada en el cable dice «los 8 bytes siguientes son un double». Lo único que permite al receptor interpretar el flujo es el *acuerdo* sobre el orden y los tipos de lo escrito: el protocolo. Si el que escribe y el que lee no coinciden, nadie te avisa: obtienes basura, una excepción o un programa bloqueado para siempre. En la Parte 1 lo provocarás a propósito.

Orden de bytes: los flujos con tipo (ingl. *data streams*) escriben todo valor de varios bytes en big-endian («orden de red») y las cadenas como UTF-8 modificado con prefijo de longitud (`writeUTF`). Dos JVM sobre arquitecturas distintas se entienden entre sí sin que tú tengas que hacer nada: el orden de los bytes es uno de los problemas clásicos del marshalling, y aquí lo resuelve la biblioteca por ti.

Petición de la parte 1: lo que escribe TypedEchoClient, byte a byte



33 bytes en total. Nada en el flujo dice dónde acaba un campo:
sólo el orden acordado (el protocolo) permite leerlo.

Figura 1. Los 33 bytes que escribe el cliente de la parte 1, en orden. Los flujos con tipo escriben los números en big-endian, y writeUTF antepone a la cadena su longitud en 2 bytes.

2. Parte 1 · Eco con tipos (calentamiento, ~30 min)

Abre el proyecto esqueleto en IntelliJ IDEA (*File → Open…* sobre la carpeta que contiene `pom.xml`; es un proyecto Maven normal, así que VS Code o Eclipse sirven igual). El paquete `echo` contiene un cliente completo y un servidor por terminar. Un intercambio por conexión:

Sentido	Contenido (¡en orden!)
Cliente → servidor	`int n  double x  UTF s`
Servidor → cliente	`int n+1  double 2·x  UTF s.toUpperCase()`

Completa los tres TODO de `TypedEchoServer`. Con el servidor en marcha, el cliente debe imprimir exactamente:

```
reply: n=42  x=3.0  s="DISTRIBUTED SYSTEMS"
```

El TODO (2) te pide llamar a dos.flush() aunque aquí no haga falta: un DataOutputStream montado directamente sobre el flujo del socket entrega cada valor en cuanto se lo das, así que no hay nada que vaciar. Los flujos con tipo no vacían ningún búfer por sí solos (a diferencia del `PrintWriter(os, true)` que conoces de las transparencias de sockets): el día que envuelvas el flujo en un BufferedOutputStream, y lo harás en cuanto te importe cuántos paquetes cuesta cada petición (mira la nota sobre Nagle al final: agrupar los valores de una petición en un solo paquete es la otra mitad de aquel remedio), la respuesta se quedará en el búfer local hasta que llames a flush(), y los dos programas se esperarán mutuamente, para siempre. Llamar a flush() al final de cada mensaje es la costumbre que te protege de eso.

Experimento E1 (hazlo y responde). En tu servidor ya terminado, intercambia la segunda y la tercera lecturas (readUTF() antes que readDouble()) y vuelve a ejecutar el cliente sin tocarlo.

> **P1.** ¿Qué ocurre, en qué extremo y por qué exactamente? Razónalo desde los bytes: el cliente escribió 8 bytes de double (valor 1,5) donde tu servidor ahora espera una cadena UTF, y `readUTF` empieza leyendo una longitud de 2 bytes. ¡Deshaz el cambio después!

El TODO (3) cierra un cabo suelto de la práctica 2: en un flujo de *texto*, que el otro extremo cierre se manifiesta como `readLine() == null` (recuerda la rareza del «echo: null»). En un flujo *binario* no hay null: `readInt()` lanza `EOFException`. Ninguna de las dos cosas es un error: así es el final de la conversación, y tu código debe esperarlo.

3. Parte 2 · El protocolo del Banco Acme (~60 min)

El Banco Acme de la práctica 0 (la clase CuentasBancoAcme) pasa a ser distribuido: las cuentas viven ahora en un servidor y los clientes operan sobre ellas por TCP. El servidor crea de antemano tres cuentas (para que las pruebas de todos se comporten igual). La práctica 0 exigía que una cuenta naciera con saldo mayor o igual que cero y te pedía no controlar errores; aquí la regla se completa: un saldo nunca baja de cero, y el servidor rechaza la operación que lo dejaría por debajo.

accountId	Titular	Saldo inicial
1	Ada	1000,00
2	Grace	25,50
3	Edsger	0,00

3.1 El protocolo

Toda petición empieza con un **código de operación** (un `int`) que dice qué operación viene a continuación: el mismo patrón que encontrarás en cada peldaño del Laboratorio 2 y en el código generado por gRPC al final del curso. El contrato completo:

Opcode	Operación	Carga de la petición (tras el opcode)	Respuesta
0	`QUIT`	(ninguna)	(el servidor cierra la sesión)
1	`BALANCE`	`int accountId`	`boolean found, double balance`

Opcode	Operación	Carga de la petición (tras el opcode)	Respuesta
2	`DEPOSIT`	`int accountId, double amount`	`boolean ok, double newBalance`
3	`WITHDRAW`	`int accountId, double amount`	`boolean ok, double balance`
4	`TRANSFER` *(opcional)*	`int from, int to, double amount`	`boolean ok, double newFromBalance`

Semántica: `BALANCE` sobre una cuenta desconocida responde `found = false` (el saldo, entonces sin significado, se envía como 0.0). `DEPOSIT` rechaza cantidades ≤ 0; `WITHDRAW` rechaza además si el saldo es insuficiente; ambos responden siempre con el saldo de la cuenta tras el intento. Todos los valores viajan exactamente en el orden de la tabla: **la tabla es el protocolo.**

Modelo de conexión: una conexión TCP por sesión de cliente. El proxy la abre una vez, envía por ella cuantas peticiones quiera y `QUIT` la termina. (Los primeros peldaños del Laboratorio 2 usan el modelo contrario, una conexión por llamada; allí compararás los dos.)

```
petición  BALANCE(1):  00 00 00 01 | 00 00 00 01
   int opcode + int accountId = 8 bytes
respuesta found = true, balance = 1000.0:
   01 | 40 8F 40 00 00 00 00 00   (boolean + double = 9 bytes)
```

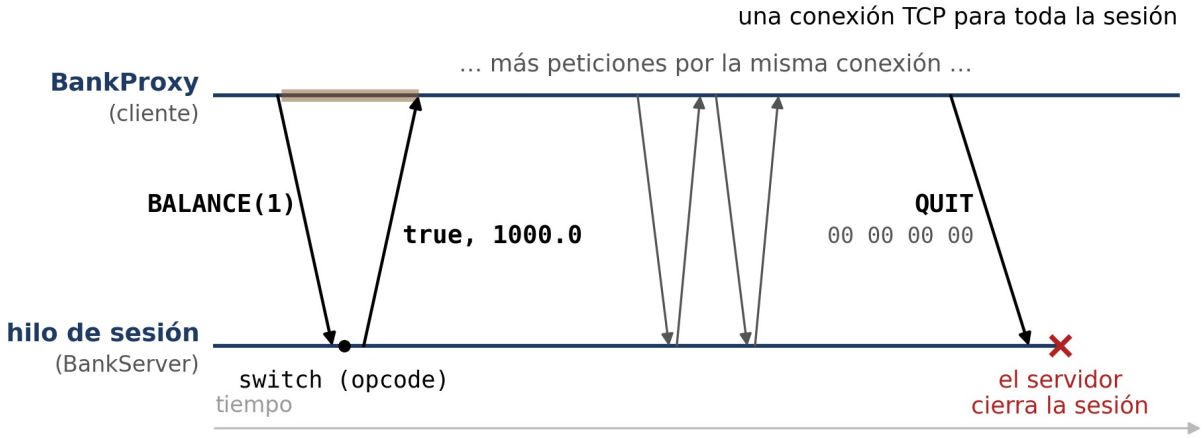


Figura 2. Una sesión con el Banco Acme: el proxy abre una conexión, envía por ella sus peticiones, cada una encabezada por su opcode, y la termina con QUIT. Los bytes son los de getBalance(1), el método que se da como modelo.

3.2 Tus tareas

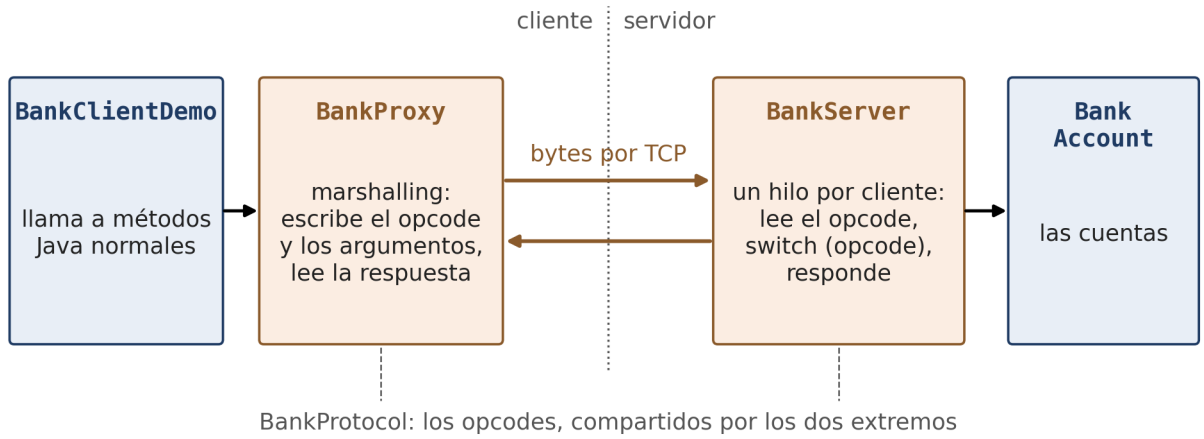


Figura 3. Quién hace qué. Sólo BankProxy y el hilo de sesión de BankServer conocen el formato de los mensajes; el resto del código llama a métodos Java normales.

- Lee primero BankProtocol: los opcodes viven en una clase compartida por cliente y servidor, que es la manera más barata de que los dos extremos respeten el contrato.
- Servidor (`BankServer`): el bucle de accept y la estructura de un hilo por cliente son los de la práctica 3 y ya están escritos. El `switch` de la sesión despacha según el opcode; el manejador `balance()` se da como modelo. **Escribe deposit() y withdraw().**
- Cliente (`BankProxy`): el proxy es la única clase que conoce el formato de los mensajes; el resto del cliente llama a métodos Java normales. `getBalance()` se da como modelo. **Escribe deposit() y withdraw().** Fíjate en el modelo en que la API Java del proxy no tiene por qué calcar el protocolo: getBalance convierte la pareja (found, balance) en un único valor de retorno. Y lee *los dos* valores de la respuesta aunque sólo uses uno: están en el cable, los leas o no.
- Ejecuta `BankServer` y después `BankClientDemo` (dos configuraciones de ejecución en IntelliJ). Contra un servidor recién arrancado, tu salida debe coincidir **exactamente** con esta transcripción (las dos últimas líneas, sólo tras la Parte 4 opcional):

```
balance(1)        = 1000.0
deposit(1, 250)   = true
balance(1)        = 1250.0
withdraw(2, 100)  = false
withdraw(2, 20)   = true
balance(2)        = 5.5
balance(7)        = -1.0
transfer(1->3, 500) = true
balance(3)        = 500.0
```

> **P2.** El servidor reacciona a un opcode desconocido cerrando la sesión (mira el caso `default`). ¿Por qué «ignorarlo y leer la siguiente petición» no es una opción en un protocolo binario? ¿Qué leería el servidor exactamente a continuación?

4. Parte 3 · Fallos y concurrencia (~30 min)

(a) Plazos de espera: tu primer detector de fallos

El constructor del proxy llama a `socket.setSoTimeout(3000)`: cualquier lectura que espere más de 3 s aborta con `SocketTimeoutException`. Pruébalo: arranca el servidor con el argumento de programa `--slow` (BALANCE tarda entonces 5 s) y ejecuta la demo.

> **P3.** Cuando salta el plazo, ¿qué sabes realmente del servidor: caído, sobrecargado o inalcanzable? ¿Cuáles de esas situaciones puedes distinguir desde tu lado del cable? Guarda tu respuesta: lo que has descrito es la incertidumbre del modelo de sistema del Tema 2, y el detector de fallos de ese mismo tema se construye sobre esta idea del plazo de espera.

(b) Dos clientes, una cuenta: vuelve la condición de carrera

El servidor dedica un hilo a cada cliente (práctica 3), así que dos sesiones pueden operar sobre la misma cuenta a la vez, y Account.deposit() es una secuencia leer-modificar-escribir. En la práctica 1 demostraste lo que eso significa. Haz lo siguiente, en orden:

- 1. Ejecuta `BankStressDemo` contra un servidor recién arrancado: dos hilos cliente ingresan 1,00 cada uno, mil veces, en la cuenta 3. Anota el saldo final: casi siempre el total esperado exacto, o a unas pocas unidades de él.

> **P4.** Account no tiene ningún synchronized y aun así la prueba sale exacta, o le faltan unas pocas unidades (ejecútala varias veces). ¿*Demuestra* eso que el servidor es seguro frente a hilos? ¿Por qué no? (Pista: ¿cuánto tiempo pasa cada ingreso dentro de la sección crítica, comparado con el viaje de ida y vuelta entre dos peticiones del mismo cliente?)

- 2. Haz visible la ventana oculta: en `Account.deposit()`, descomenta la línea marcada pause(1), que hace el papel de un paso lento (digamos, escribir un registro de auditoría) entre leer el saldo y escribirlo. Vuelve a ejecutar la demo: ahora los ingresos se pierden de forma espectacular, y distinta en cada ejecución.

- 3. Arréglalo como arreglaste el Contador en la práctica 1 (`synchronized` en deposit, withdraw y getBalance) y repite con la pausa aún puesta: exacto otra vez, sólo que más lento, porque el cerrojo serializa los ingresos.

La lección va mucho más allá de los bancos: las ventanas de carrera duran microsegundos, y el ritmo petición-respuesta de los servidores de red las esconde durante las pruebas. Se disparan en producción, bajo carga, cuando por fin dos peticiones caen lo bastante juntas. **Una prueba que pasa demuestra que existen ejecuciones correctas, no que no haya carreras.**

5. Parte 4 (opcional) · TRANSFER, o dos cerrojos y una trampa

Implementa el opcode 4 de extremo a extremo: `Bank.transfer()`, el manejador del servidor y el método del proxy (los TODO están marcados). Una transferencia debe ser atómica sobre dos cuentas: la retirada y el ingreso deben ocurrir como un único paso indivisible, así que necesitas los dos cerrojos a la vez. Antes de escribirla, piensa qué puede pasar si dos transferencias A→B y B→A toman cada una su cerrojo «from» y esperan después el del otro. El TODO contiene una pista (toma los cerrojos en un orden fijo); la trampa tiene nombre clásico: **interbloqueo (deadlock)**, y volverás a encontrarla en la teoría.

6. Qué viaja por el cable (y qué cae en el examen)

> **P5.** Escribe, byte a byte, la petición que viaja del cliente al servidor en la llamada `deposit(2, 20.0)`: para cada campo, su tipo Java, su valor y su tamaño en bytes, en el orden del cable, más el tamaño total. Haz lo mismo con la respuesta. (Éste es exactamente el tipo de pregunta que el examen de laboratorio hace sobre protocolos como éste.)

Destrezas que este peldaño te da para el PR-EX-1: leer y escribir una tabla de opcodes y formatos de mensaje; implementar a mano el marshalling y el unmarshalling a partir de esa tabla; el patrón proxy + despachador; y la semántica de `flush()`, `EOFException` y `setSoTimeout()`.

7. Entregables

- Un zip de tu proyecto Maven (tu código de las Partes 1–3, más la 4 si la has hecho).
- Tus respuestas a P1–P5 en un breve `ANSWERS.md` en la raíz del proyecto (unas líneas cada una; razonadas, no un simple «sí/no»).

Sube el zip a Moodle antes de la fecha límite indicada en la página del curso. La entrega es voluntaria y no lleva nota: sirve para que el profesor te la corrija. El trabajo es individual; discutir ideas está bien, compartir código no.

Notas al pie para curiosos

Por qué el código activa TCP_NODELAY. Nuestros mensajes son diminutos (unos pocos bytes cada uno). El algoritmo de Nagle de TCP retrasa los paquetes pequeños con la esperanza de agruparlos y, combinado con los ACK retardados del receptor, puede costar decenas de milisegundos por viaje de ida y vuelta (los tiempos exactos dependen del sistema operativo). En nuestras pruebas, con él la demo de estrés tarda más de un minuto; con `setTcpNoDelay(true)`, un cuarto de segundo. Los protocolos petición-respuesta con mensajes pequeños desactivan Nagle de forma rutinaria; los frameworks de RPC reales hacen lo mismo.

¿Dinero en un double? Vale para esta práctica, nunca en un banco real: la coma flotante binaria no puede representar con exactitud la mayoría de las cantidades decimales. Los sistemas reales usan céntimos enteros o `BigDecimal`.